

Harness Engineering, SDD y 8090 — Notas para la demo

Documento de referencia armado a partir de la conversación. Objetivo: entender qué es un "harness" en el contexto de SDD y 8090, y preparar una demo de 15 minutos que implemente un harness sobre un CRUD service en C#.

1. La palabra "harness" tiene varios significados

"Harness" viene del testing de software: es el *andamiaje* que rodea a algo para hacerlo funcionar o para probarlo. En IA se usa en sentidos distintos, y por eso el pedido "creá un harness" es ambiguo hasta que se aclara el contexto:

- **Agent harness** — el orquestador que envuelve a un LLM y lo convierte en un agente que ejecuta cosas (loop de razonamiento-acción, tool-calling, manejo de contexto, reintentos, aprobaciones, acceso a archivos/shell). En el ecosistema Microsoft esto se volvió un concepto de producto: la clase `HarnessAgent` / `AsHarnessAgent()` del Microsoft Agent Framework (1.0 GA, abril 2026).
 - **Evaluation harness** — el andamiaje para medir/testear sistemáticamente el comportamiento de un agente (casos de prueba, métricas, comparar versiones, CI/CD). En .NET: `Microsoft.Extensions.AI.Evaluation`.
 - **Test harness clásico** — infraestructura de testing tradicional (mocks, fixtures, stubs), sin nada específico de IA.
 - **Harness engineering** — *este es el que importa para la demo*. Ver abajo.
-

2. El trío que importa: SDD → Harness → 8090

Estos tres conceptos cuentan una sola historia.

SDD (Spec-Driven Development)

Método donde **la especificación es el artefacto principal de ingeniería y el código es su expresión**. En vez de escribir código que implica un diseño, se escribe un diseño que produce el código. Como los agentes de IA ya pueden ejecutar specs directamente, la spec pasa a ser el artefacto durable (lo que los humanos mantienen, revisan e iteran) y el código es lo que el agente genera.

Distingue dos tipos de sesión:

- **Planning session:** humano + agente producen la spec. No se escribe código. Salida = documento de spec.
- **Execution session:** el agente toma la spec aprobada y la implementa.

Harness (harness engineering)

El harness no es una herramienta: es el *sistema de controles* alrededor del agente de coding. Es la práctica de diseñar guías hacia adelante (feedforward) y sensores de feedback para que el código esté continuamente regulado hacia el estado que definen las specs. Tres capas:

1. **Feedforward** — specs, constraints, guardrails que se le dan al agente *antes* de que corra.
2. **Runtime** — orquestación de tools, reintentos, observabilidad.
3. **Feedback** — tests automáticos, linters, chequeos estructurales, gates de revisión que corren *después*.

Idea clave para el white paper: SDD es lo que hace *practicable* al harness. Sin specs, el harness no tiene nada concreto que hacer cumplir.

Referencia conceptual: Martin Fowler, "Harness engineering for coding agent users".

Encuadre de trilogía: Fundamentos (qué es un agente, cómo manejarlo) → SDD (cómo estructurar el trabajo para que el agente lo ejecute bien) → Harness engineering (cómo ingenierizar el sistema alrededor del agente). El harness es el nivel más avanzado.

8090

Es la versión productizada y comercial de todo esto. Se presentan como la plataforma de desarrollo de software "AI-native" que lleva del **spec al deploy** con calidad, control y consistencia, apuntada a industrias reguladas (salud, finanzas, gobierno) y respaldada por un partnership con una Big Four. Bandera que levantan: que el líder de negocio defina en lenguaje natural qué se construye antes de escribir una línea de código, para que no sean los agentes ni los devs junior los que tomen decisiones de arquitectura. Es SDD + harness convertido en plataforma con auditabilidad.

3. La demo de 15 minutos: "el contraste"

La demo que mejor comunica el concepto y es realista de armar en una semana:

1. Agarrar una tarea chica y acotada (un endpoint o un CRUD service).
2. Mostrar al agente **sin harness** (vibe coding) y cómo driftea o viola algo.
3. Mostrar la misma tarea **con harness**: spec con criterios de aceptación y de verificación + gates automáticos (tests derivados de la spec, linter, chequeo estructural de arquitectura).
4. **Momento clave:** inyectar una violación (o un cambio en la spec) y mostrar el gate cazándola y el loop autocorrigiéndose.

Mensaje: *no estás revisando cada respuesta, estás diseñando las condiciones para que el resultado sea correcto.*

Reparto de tiempo sugerido (15 min):

- ~3 min de concepto (SDD → harness → dónde entra 8090)
 - ~8 min de demo en vivo
 - ~4 min de "esto es lo que el harness cazó que el prompteo crudo no"
-

4. Qué construye cada quién (el reparto)

- Los `.md` (specs) = la capa de feedforward. Lo que el agente lee antes de escribir código. Es la parte SDD, el artefacto durable.
- El código C# (gates) = la capa de feedback. El corazón del harness. Son los sensores que verifican que lo generado cumple la spec.
- El código de la app = lo genera el agente, en vivo. `/src` arranca vacío. Que ese código aparezca solo y pase los gates es la demo.
- Orquestación mínima — un script que encadena: correr agente → correr gates → si falla, devolver el error al agente → repetir.

Reparto de esfuerzo real: ~30% escribir buenas specs, ~70% escribir gates que atrapen violaciones de verdad. Un gate que no atrapa nada no demuestra nada.

5. Brief para pegar en Claude dentro de VS Code

Contexto: Estoy armando una demo de "harness engineering" sobre Spec-Driven Development (SDD). El harness es la capa de control alrededor de un agente de coding: specs (feedforward) + gates automáticos (feedback). Vos vas a construir el ANDAMIAJE del harness, NO el código de la aplicación.

Objetivo: Un CRUD service de "Products" (REST API en ASP.NET Core, .NET 8) usado como caso de prueba del harness.

Generá esta estructura:

`/specs`

`constitution.md` → principios y guardrails globales: convenciones de naming, arquitectura en capas obligatoria (Api → Application → Domain, Infrastructure sólo referenciada por Application), prohibiciones (ej: los controllers NO acceden al DbContext directo), manejo de errores estándar.

`products-spec.md` → la feature spec: el QUÉ, no el CÓMO. Entidad Product (Id, Name, Price, Stock), endpoints CRUD (GET all, GET by id, POST, PUT, DELETE), reglas de negocio

(Price > 0, Name requerido y único, Stock >= 0),
y edge cases (id inexistente -> 404, validación
fallida -> 400 con detalle).

verification.md -> criterios de aceptación en forma chequeable, mapeados
1:1 a los tests del gate.

/gates (ESTO SÍ lo escribís vos, es el corazón del harness)

- Proyecto de tests xUnit con casos derivados de verification.md (uno por criterio de aceptación). Los tests apuntan a la API que el agente va a generar después en /src.
- Reglas de arquitectura con NetArchTest que hagan cumplir la constitution: Controllers no dependen de EF Core / DbContext; Domain no depende de Infrastructure; nombres de controllers terminan en "Controller".
- .editorconfig con analyzers de Roslyn en modo warning-as-error para las reglas clave.

/scripts

verify.sh (o verify.ps1) -> corre `dotnet build` + `dotnet test` y devuelve
exit code distinto de cero si algún gate falla.
Este es el loop de feedback.

/src -> DEJALO VACÍO con un README.md que diga: "El código de la app se genera
en vivo a partir de /specs. No implementar todavía."

Reglas:

- NO implementes la API en /src. Eso es la demo en vivo.
- Los gates tienen que poder compilar y correr aunque /src esté vacío (que fallen esperablemente por falta de implementación está bien: mostralo).
- Mantené todo mínimo y legible: es para explicar en 15 minutos.
- Al final, escribí un DEMO.md con el guion: (1) mostrar specs, (2) pedirte a vos implementar /src desde la spec, (3) correr verify, (4) inyectar una violación de arquitectura y mostrar el gate cazándola.

Notas de uso

- Después de que arme el andamiaje, la **demo en vivo** es una segunda instrucción: *"implementá (/src) cumpliendo (/specs/products-spec.md) hasta que (verify) pase en verde"*. Ahí se ve el harness trabajando.
- El gate que más impacta es el de **NetArchTest**. Para el cierre, metés a mano una violación obvia (un controller que inyecta el `DbContext` directo), corrés `verify`, y el gate la caza. Ese es el "click" que hace entender qué es un harness.

6. Referencias

- Martin Fowler — *Harness engineering for coding agent users*
- Loiane Groner — *Harness Engineering: The Missing Layer in Specs-Driven AI Development*
- arXiv — *Spec-Driven Development: From Code to Contract in the Age of AI Coding Assistants* (Feb 2026)
- Augment Code — *What Is Spec-Driven Development? A Complete Guide*
- 8090 — <https://www.8090.ai/>
- GitHub Spec Kit (Microsoft) — toolkit de SDD
- Guía SDD (jmlopezdona) — *the intermediate course between AI coding agent fundamentals and harness engineering*